Zed for Windows
It's here—fast, native
Download
Learn More

Sponsored by Dometrain Courses—Get 30% off **Dometrain Pro** with code **ANDREW30** and access the best courses for .NET Developers

December 09, 2025 ~11 min read
IDES VS CODE

Trying out the Zed editor on Windows for .NET and Markdown

Share on:

In this post I provide my initial thoughts about the Zed editor. I provide my first impressions, some of the customizations I made to get comfortable with it, and my conclusions on using it for working with .NET and Markdown documents.

What's wrong with VS Code?

Andrew Lock | .Net Escapades
Want an email when there's new posts? Subscribe

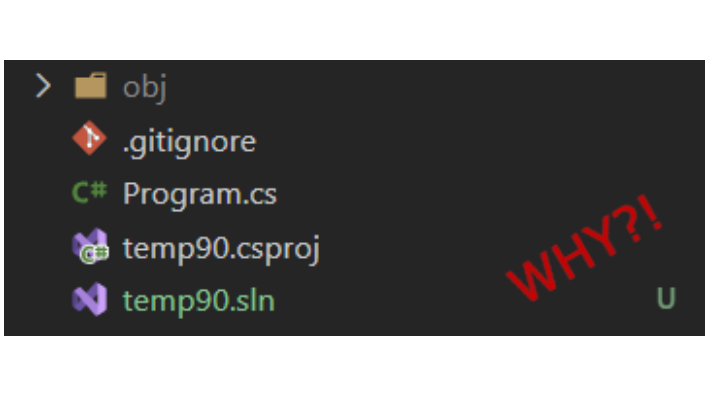
Before I get to my actual impressions, I think it's probably worth discussing *why* I started looking at the Zed editor in the first place. After all, I currently have JetBrains Rider, Visual Studio, *and* VS Code installed on my laptop, do I *really* need another one? 😅

Of course, while you *can* use all these IDEs/editors for much of the same tasks, they have different sweet spots. For example, JetBrains Rider is my go-to IDE day to day. It's what I use for virtually all my "real" development these days. Visual Studio fills essentially the same role for me, and consequently I don't use it much, there's just occasional things where I find the deeper integration with Azure (among other things) come in handy.

And then we have VS Code. VS Code always *used* to be a quick, lightweight editor. It had good-enough general syntax highlighting for all sorts of arbitrary languages, the .NET integration was "good enough" when all I want to do is hack together a quick console app for a blog post, and it's actually a pretty good markdown editor. But VS Code has been starting to bug me more and more recently. It's nothing that I can really put my finger on, it's the little things…

First of all, and the main gripe to be honest, is that VS Code just doesn't feel as snappy as it used to. It's not terrible, but I go out of my way to try to *not* have too many extensions installed now, because it just exacerbates things. I want to be able to edit a file in explorer and have it pop up straight away, not to have to wait 5 seconds for the window to appear.

The other thing that bugs me recently is, ironically, the "improved .NET support", by way of the C# Dev Kit. And the experience that irritates me is the same irritating feature Visual Studio insists on: creating *sln* files all over the place. There's a perfectly good *csproj* in the folder, that's good enough! Stop creating pointless *sln* files!



Obviously I understand *why* it does this, but I literally never want this behaviour. I also don't care about the "solution explorer" window—if I need that view I'll use a real IDE like Rider. Why can't you just be happy as an editor, VS Code?!

I have some other more minor gripes, but as none of that's particularly productive, let's move on. One thing that I *do* enjoy about VS code is the markdown editing experience. I use the Markdown All in One extension, Spell Right, and Word Count, and it does everything I need.

So in conclusion I was feeling a bit down on VS Code as an editor. It was still better than anything I had used before, but it wasn't quite the darling that I had once considered it to be. And that's when a colleague at work mentioned they'd been using (and enjoying) Zed.

Maybe Zed can replace VS Code?

Zed is an editor I'd heard about a few times on the changelog podcast. It was interesting, as it was created by Nathan Sobo, a developer that previously worked on Atom at GitHub, but it went in a completely different direction. While Atom used Chromium and Node (and spawned the Electron framework so many apps use today), Zed was very different. It was built in Rust, and designed to be very fast.

Zed was interesting from an academic point of view, but not hugely practical for me, for several reasons:

1. It didn't support Windows, (until recently)!
2. A big sell is about being "collaborative", something I wasn't interested in for an editor.
3. They've largely pivoted Zed to be an AI play, again something I'm not interested in for an editor.

But then, in October 2025, Zed released a Windows build. After a colleague at work told me how much they were enjoying it, I decided to give it a go, and see if it could replace VS Code for my day to day usages.

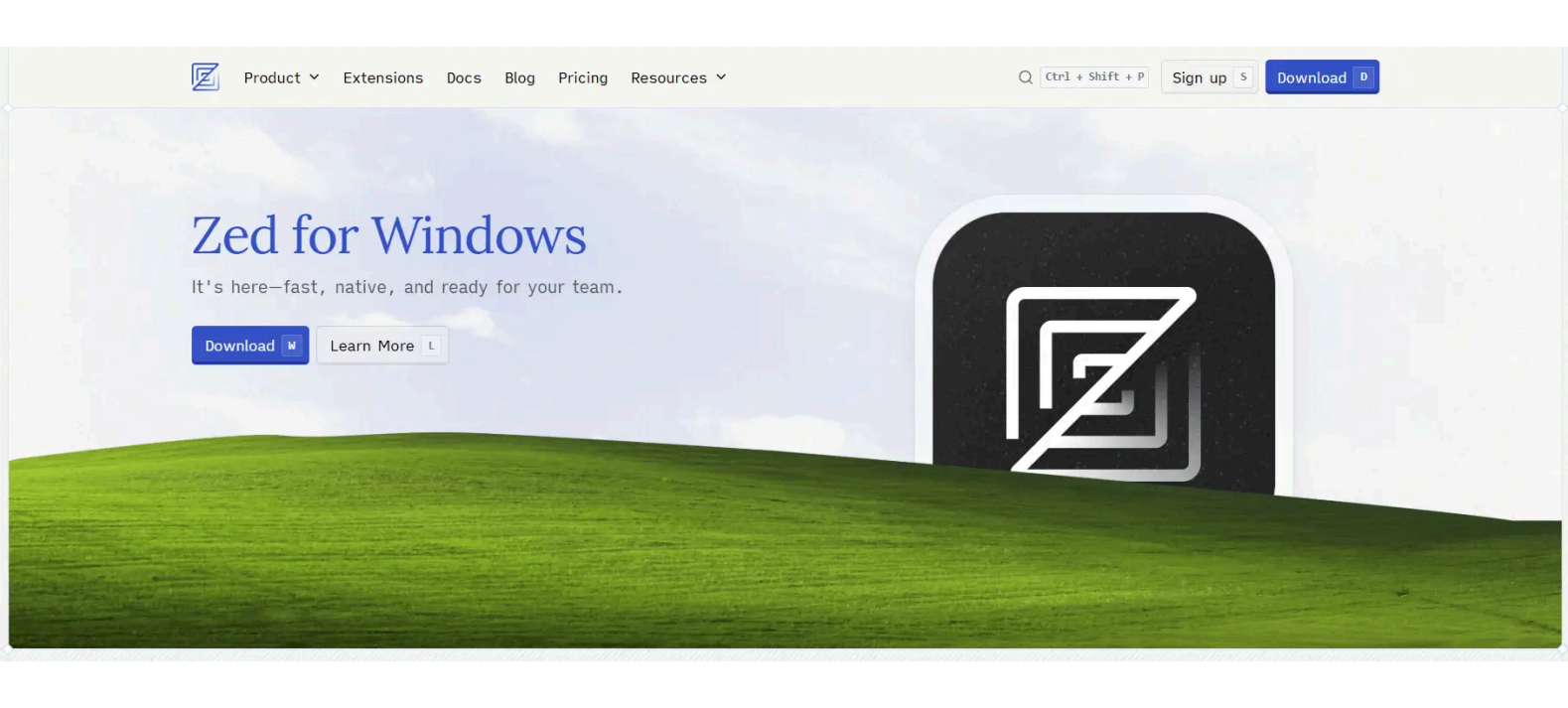
As a reminder, what I really wanted was and editor that:

- Is *fast*.
- Works well with Markdown
- Works with small .NET projects (when opening an IDE is too slow!)

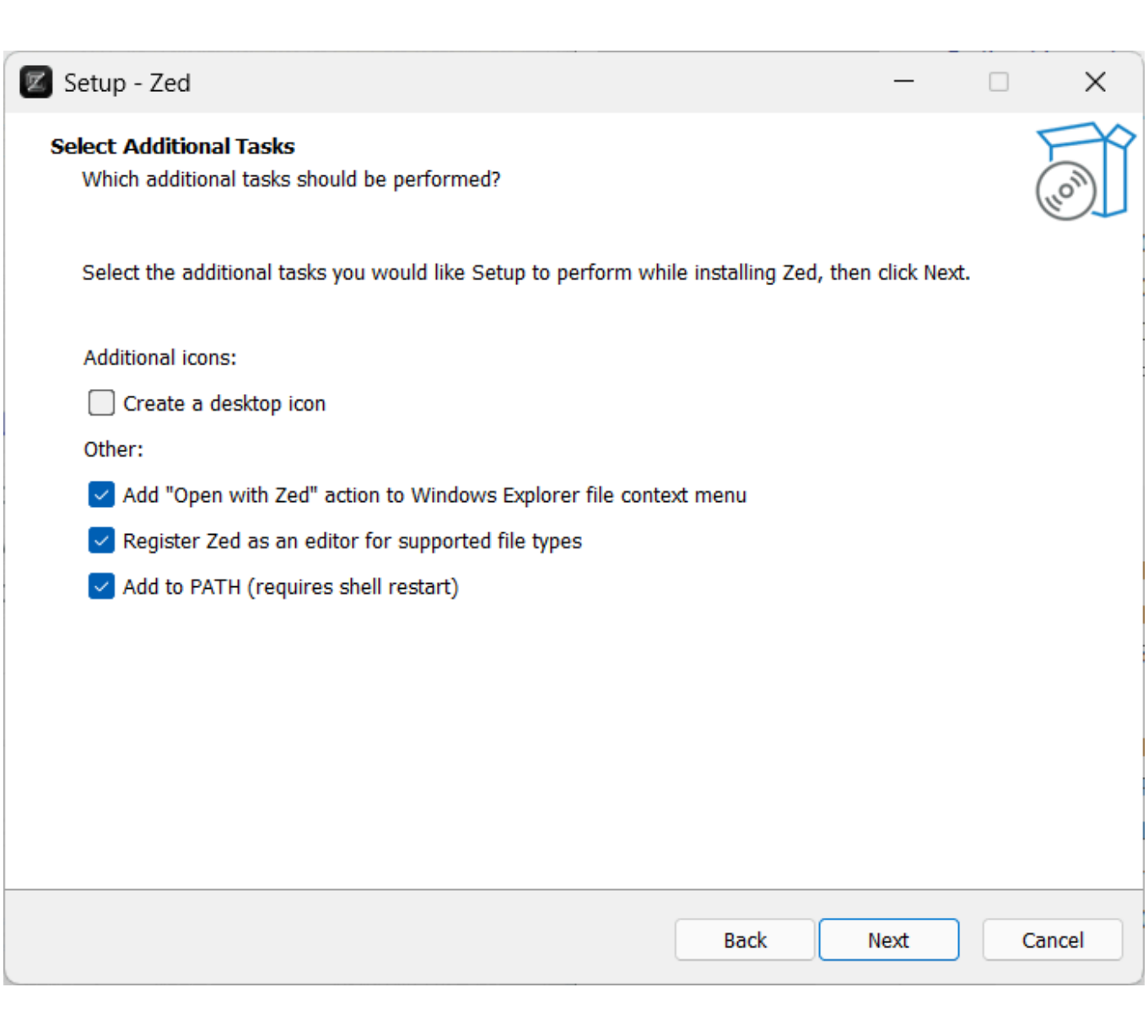
So I installed Zed onto my personal machine, and gave it a try!

Installing Zed for Windows

To install Zed, head to https://zed.dev/windows, and hit the download button (Or just press `W`!)



This downloads the installer, which is your pretty standard affair, and it includes the usual options for adding Zed to the explorer context window for example (ideal for what I would be using it for).

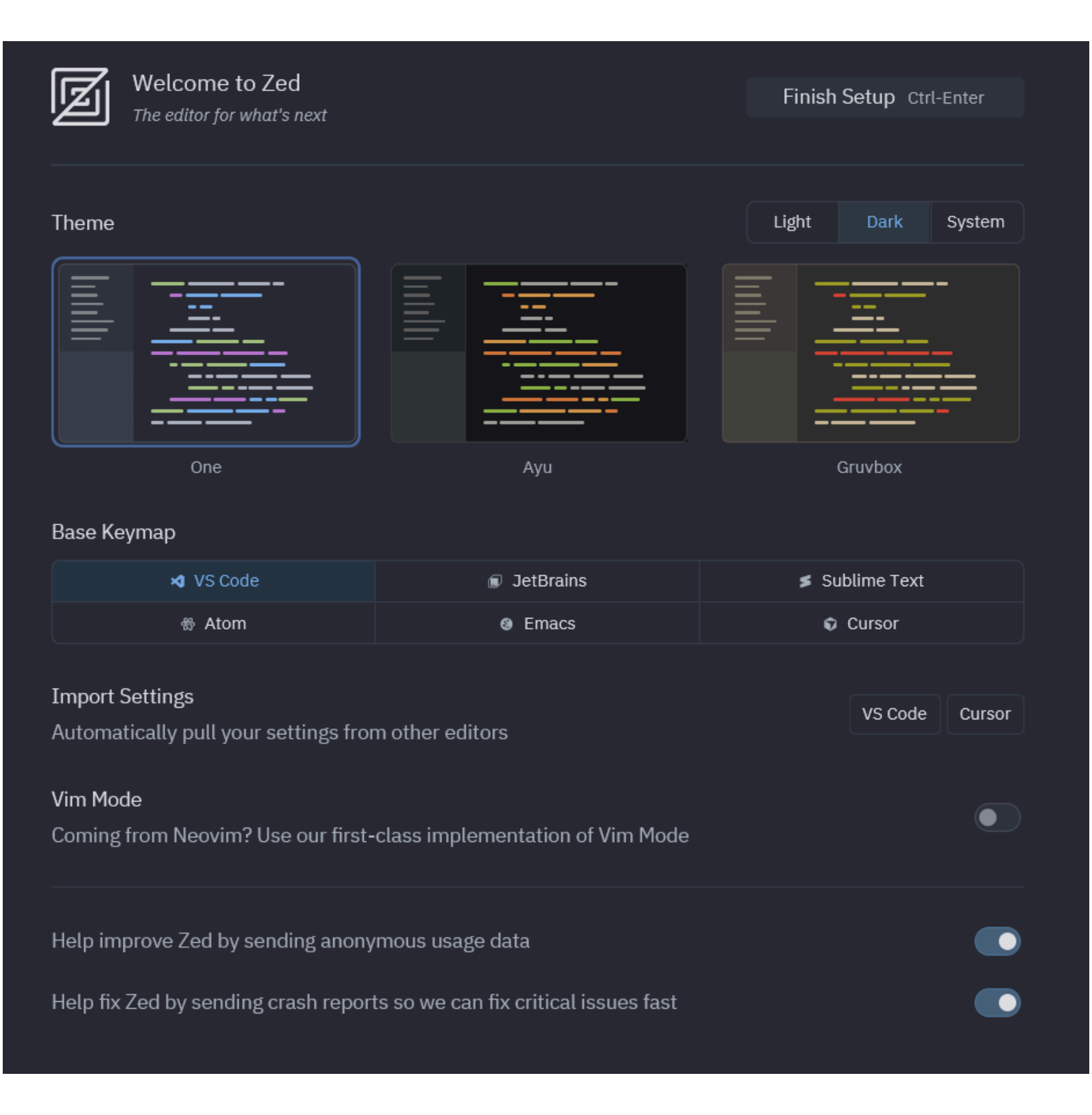

Setup - Zed

Select Additional Tasks
Which additional tasks should be performed?

Select the additional tasks you would like Setup to perform while installing Zed, then click Next.

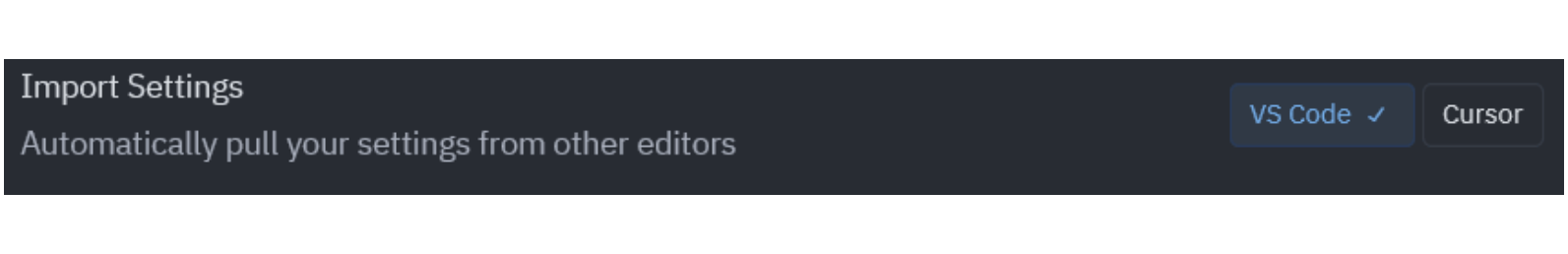
Additional icons:
☐ Create a desktop icon
Other:
☑ Add "Open with Zed" action to Windows Explorer file context menu
☑ Register Zed as an editor for supported file types
☑ Add to PATH (requires shell restart)

On running Zed for the first time, you're presented with some base customization options:

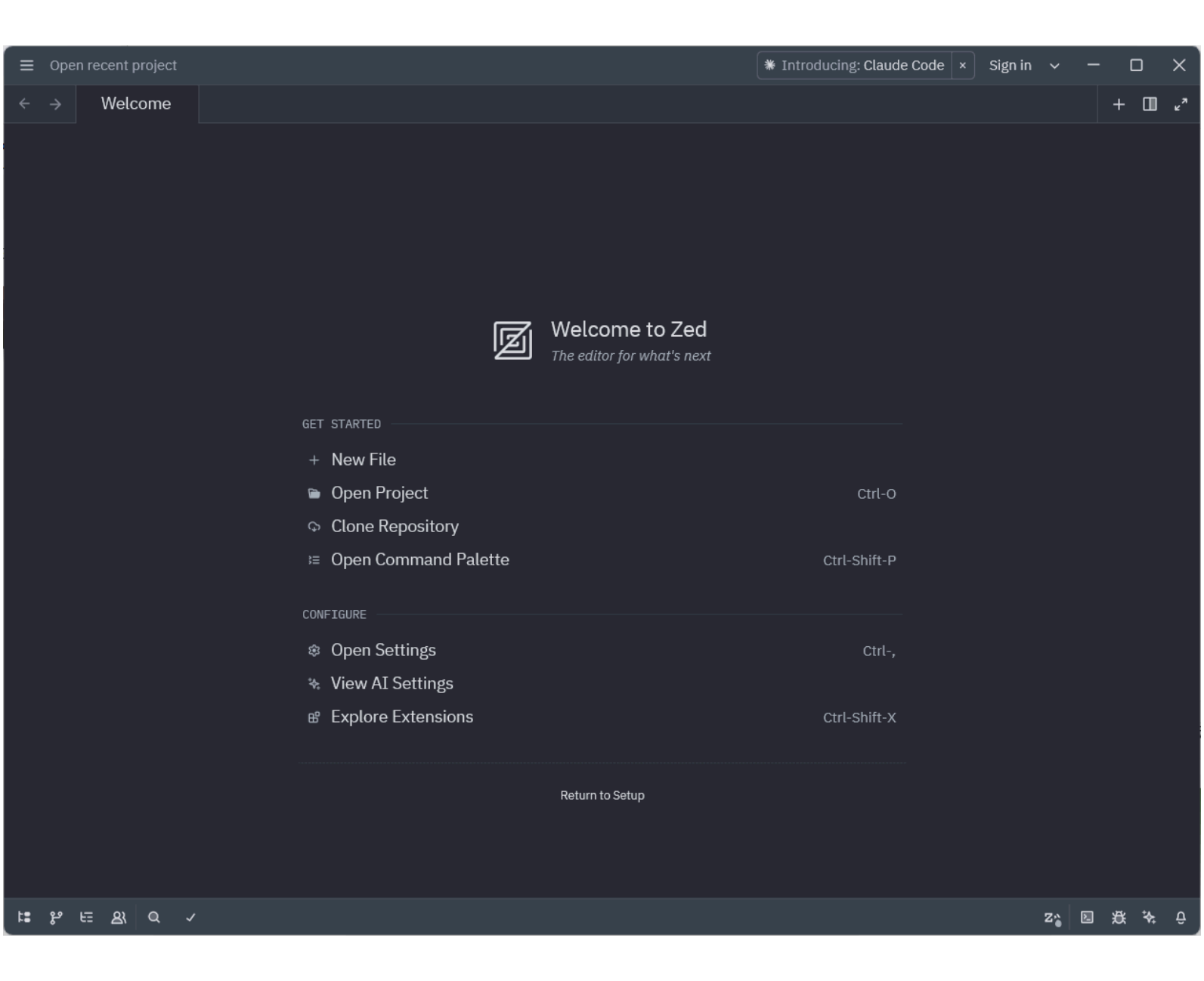

Welcome to Zed
The editor for what's next

Theme: Light / Dark / System — One, Ayu, Gruvbox
Base Keymap: VS Code, JetBrains, Sublime Text, Atom, Emacs, Cursor
Import Settings — Automatically pull your settings from other editors
Vim Mode — Coming from Neovim? Use our first-class implementation of Vim Mode
Help improve Zed by sending anonymous usage data
Help fix Zed by sending crash reports so we can fix critical issues fast

The ability to base your keymap on other well known editors *and* to import settings from other editors is a very neat onboarding trick. I chose to import my settings from VS Code:


Import Settings
Automatically pull your settings from other editors

Once you're done with the setup, you're dropped into the default welcome screen, which overall looks pretty similar to the VS Code experience. You have your getting started actions, an explanation of the shortcuts available, and a bunch of toolbars around the place:


Welcome to Zed
The editor for what's next

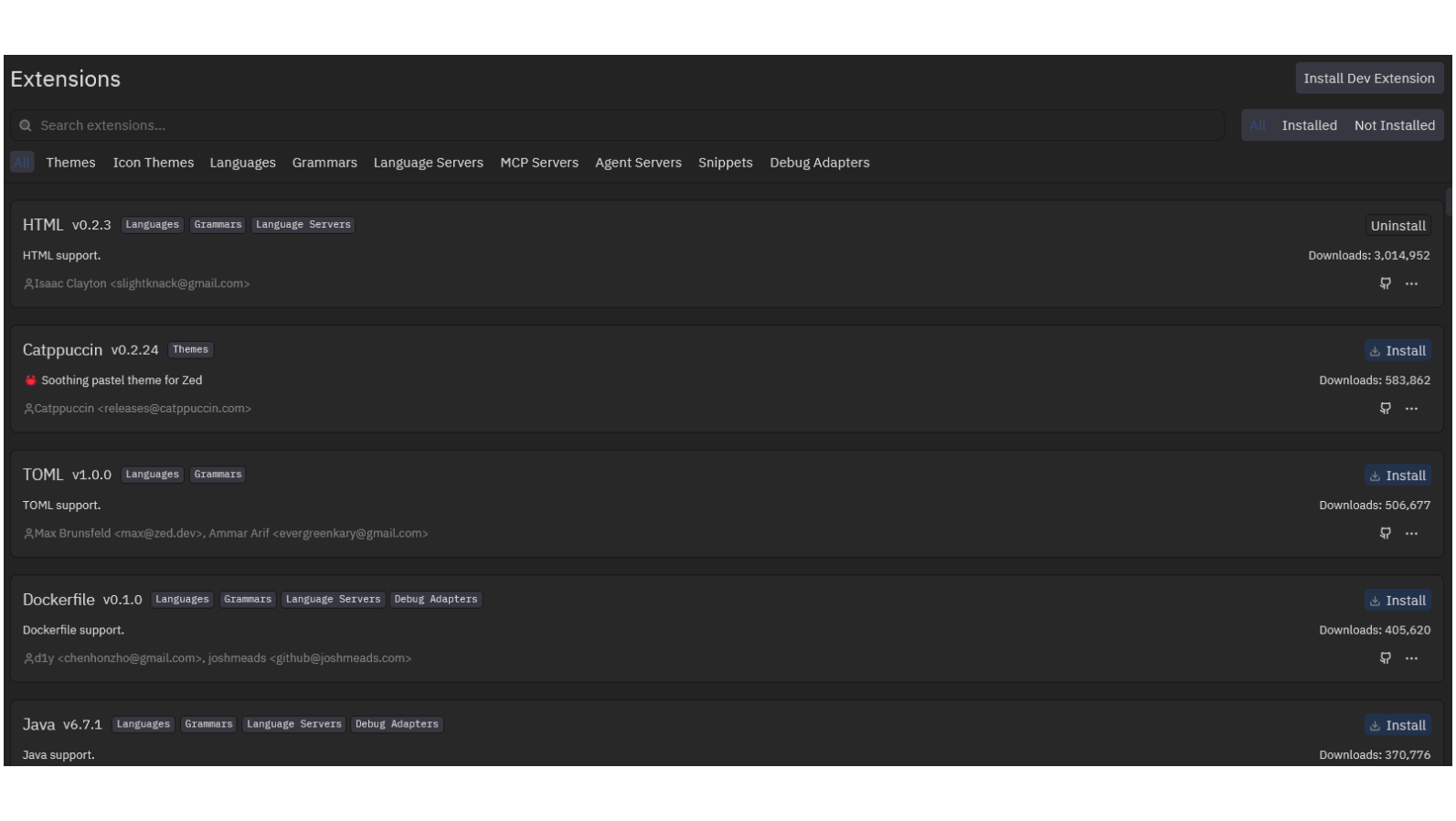
GET STARTED
- New File
- Open Project Ctrl-O
- Clone Repository
- Open Command Palette Ctrl-Shift-P

CONFIGURE
- Open Settings Ctrl-,
- View AI Settings
- Explore Extensions Ctrl-Shift-X

I tried generally just opening a few files at this point and looking around, typing a little bit, and I have to say, the first impression is very impressive. It's just *so* smooth and snappy. Seriously, it feels *so* fast compared to VS Code. It was a good start! 😄

Adding support for C# to Zed

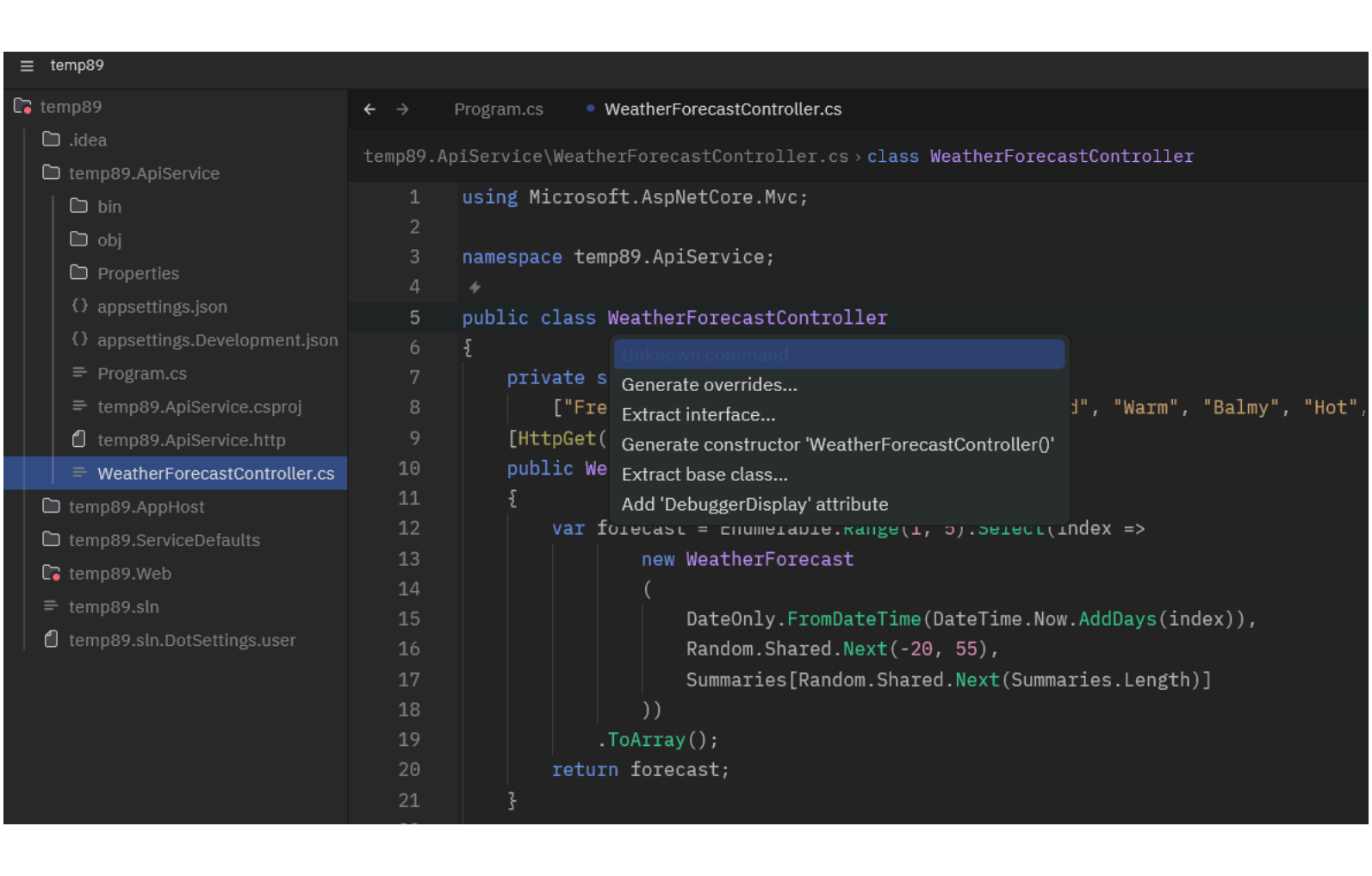
Much like VS Code, Zed is built around various protocols, and delegates large parts of its core functionality to extensions. There are extensions for themes, extensions for icon packs, extensions for languages…the list goes on! You can open the extension window using the same shortcut as VS Code, `Ctrl`+`Shift`+`X` (at least, assuming you're using the VS Code shortcuts!)


Extensions

HTML v0.2.3 Languages Grammars Language Servers HTML support. Isaac Clayton <slightknack@gmail.com>	Uninstall Downloads: 3,014,952
Catppuccin v0.2.24 Themes Soothing pastel theme for Zed Catppuccin <releases@catppuccin.com>	Install Downloads: 583,862
TOML v1.0.0 Languages Grammars TOML support. Max Brunsfeld <max@zed.dev>, Ammar Arif <evergreenkary@gmail.com>	Install Downloads: 506,677
Dockerfile v0.1.0 Languages Grammars Language Servers Debug Adapters Dockerfile support. d1y <chenhonzho@gmail.com>, joshmeads <github@joshmeads.com>	Install Downloads: 405,620
Java v6.7.1 Languages Grammars Language Servers Debug Adapters Java support.	Install Downloads: 370,776

To add support for C#, open the extensions window, search for C#, and install the C# extension by `fminkowski`. This uses the omnisharp Language Server to provide language support. This is essentially the same backing implementation as the *original* VS Code C# extension, before Microsoft moved to the C# Dev Kit extension.

Once the extension is installed you get syntax highlighting for your C# code, as well as refactoring options, just as you would expect in VS Code, though these will likely be more limited than the options available as part of the VS Code C# Dev kit.

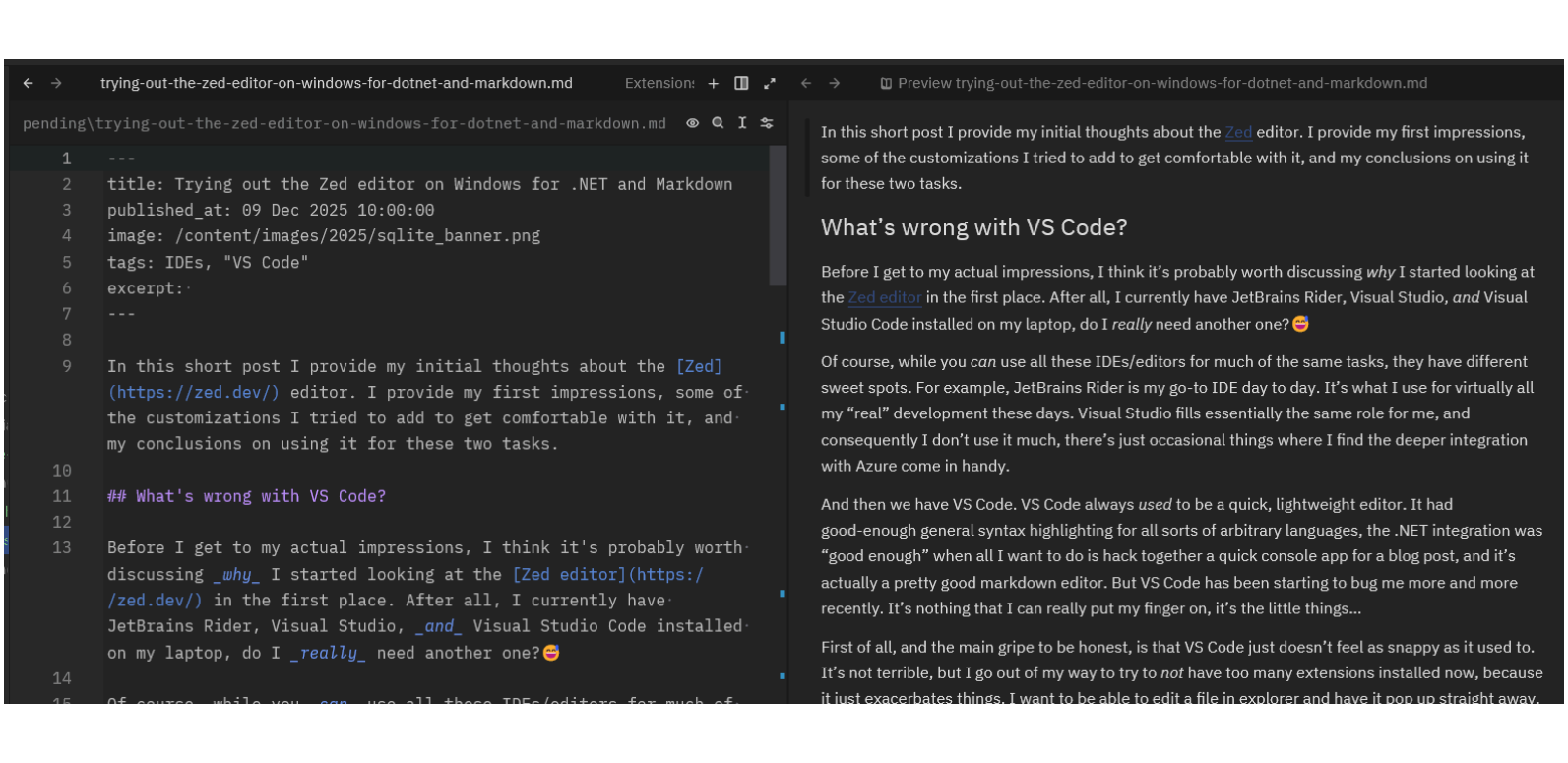

temp89.ApiService\WeatherForecastController.cs › class WeatherForecastController

```
using Microsoft.AspNetCore.Mvc;

namespace temp89.ApiService;

public class WeatherForecastController
{
    private s
        ["Fre                                          d", "Warm", "Balmy", "Hot",
    [HttpGet(
    public We
    {
        var forecast = Enumerable.Range(1, 5).Select(index =>
            new WeatherForecast
            (
                DateOnly.FromDateTime(DateTime.Now.AddDays(index)),
                Random.Shared.Next(-20, 55),
                Summaries[Random.Shared.Next(Summaries.Length)]
            ))
            .ToArray();
        return forecast;
    }
```

That's the most important part of getting .NET editing working in Zed, but I also tweaked the themes to be more familiar. I installed the JetBrains Rider theme, and also the JetBrains New UI Icon Theme, though the latter didn't seem to have special icons for C# files, so that's a bit of a shame. Anyway, that's it for editing .NET for now; let's look at markdown instead.

Adding support for Markdown

The good news is that Zed has built-in support for Markdown, so there's nothing you *need* to install. That said, it's relatively pretty bare bones:


pending\trying-out-the-zed-editor-on-windows-for-dotnet-and-markdown.md

```
---
title: Trying out the Zed editor on Windows for .NET and Markdown
published_at: 09 Dec 2025 10:00:00
image: /content/images/2025/sqlite_banner.png
tags: IDEs, "VS Code"
excerpt:
---

In this short post I provide my initial thoughts about the [Zed]
(https://zed.dev/) editor. I provide my first impressions, some of
the customizations I tried to add to get comfortable with it, and
my conclusions on using it for these two tasks.

## What's wrong with VS Code?

Before I get to my actual impressions, I think it's probably worth
discussing _why_ I started looking at the [Zed editor](https:/
/zed.dev/) in the first place. After all, I currently have
JetBrains Rider, Visual Studio, _and_ Visual Studio Code installed
on my laptop, do I _really_ need another one?😅
```

Preview trying-out-the-zed-editor-on-windows-for-dotnet-and-markdown.md

In this short post I provide my initial thoughts about the Zed editor. I provide my first impressions, some of the customizations I tried to add to get comfortable with it, and my conclusions on using it for these two tasks.

What's wrong with VS Code?

Before I get to my actual impressions, I think it's probably worth discussing *why* I started looking at the Zed editor in the first place. After all, I currently have JetBrains Rider, Visual Studio, *and* Visual Studio Code installed on my laptop, do I *really* need another one?😅

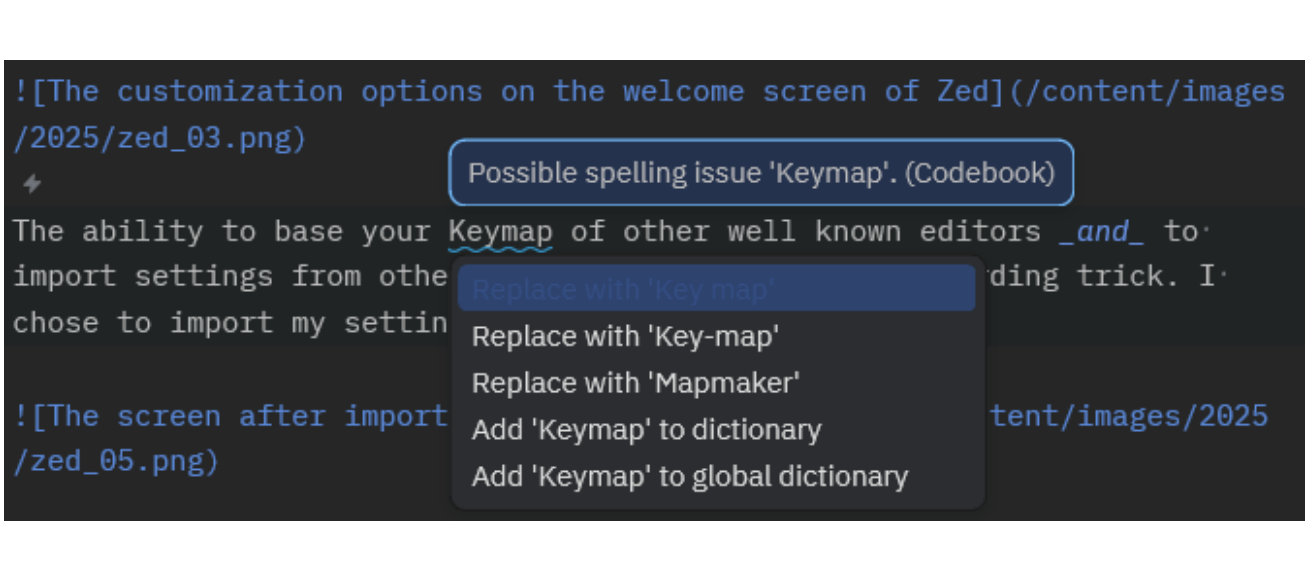
Of course, while you *can* use all these IDEs/editors for much of the same tasks, they have different sweet spots. For example, JetBrains Rider is my go-to IDE day to day. It's what I use for virtually all my "real" development these days. Visual Studio fills essentially the same role for me, and consequently I don't use it much, there's just occasional things where I find the deeper integration with Azure come in handy.

And then we have VS Code. VS Code always *used* to be a quick, lightweight editor. It had good-enough general syntax highlighting for all sorts of arbitrary languages, the .NET integration was "good enough" when all I want to do is hack together a quick console app for a blog post, and it's actually a pretty good markdown editor. But VS Code has been starting to bug me more and more recently. It's nothing that I can really put my finger on, it's the little things…

First of all, and the main gripe to be honest, is that VS Code just doesn't feel as snappy as it used to. It's not terrible, but I go out of my way to try to *not* have too many extensions installed now, because it just exacerbates things. I want to be able to edit a file in explorer and have it pop up straight away.

trying-out-the-zed-editor-on-windows-for-dotnet-and-markdown.md
Extensions

You get syntax highlighting of the markdown text, and a preview pane, but there's not a huge number of other features (more on that later). For me, the one critical extension I added was a spell checker. I added CodeBook, which is actually intended to be used as a spell checker inside *code* rather than markdown, but it does work with markdown at least:



```
![The customization options on the welcome screen of Zed](/content/images
/2025/zed_03.png)

The ability to base your Keymap of other well known editors _and_ to
import settings from othe                          ding trick. I
chose to import my settin

![The screen after import                          tent/images/2025
/zed_05.png)
```

Possible spelling issue 'Keymap'. (Codebook)
Replace with 'Key map'
Replace with 'Key-map'
Replace with 'Mapmaker'
Add 'Keymap' to dictionary
Add 'Keymap' to global dictionary

.

But the question is, did I stick with it? Did I write this post in Zed?

My first impressions after trying Zed

The short answer is, no, I didn't stick with Zed, and instead went back to VS Code. There's a lot of reasons for that which I'll get into, but before I start nipicking, I think it's worth highlighting the good bits.

The good bits

Zed feels *really* smooth to use. It opens quickly, tabbing between documents is lighting fast, and even typing feels much faster than VS Code. Switching folders/workspaces is *crazy* fast😲 The performance is one of those things that you don't realise you're missing until you try it for yourself! 😃

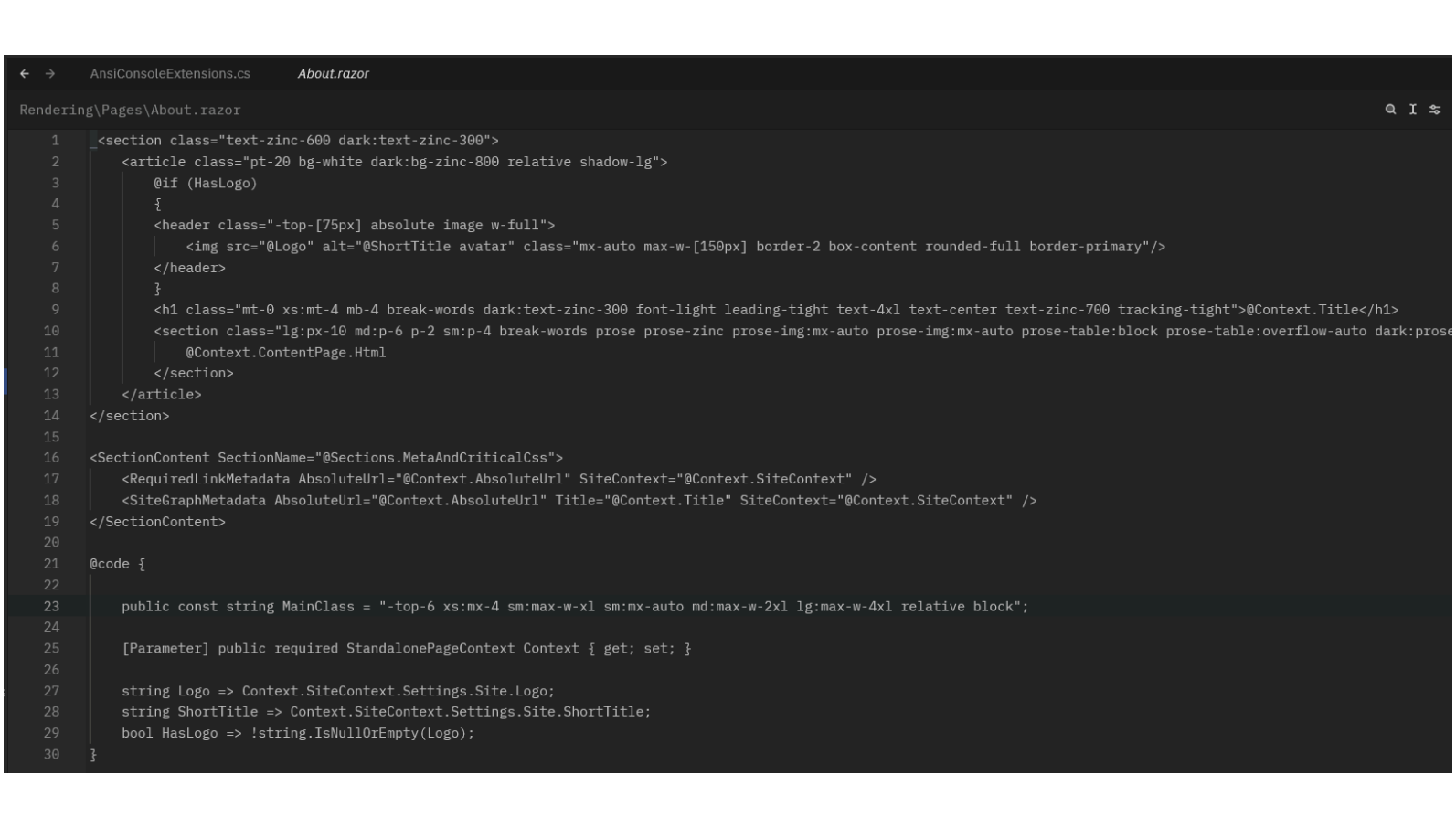
Also, I was really impressed with the care that's clearly gone into building and designing Zed. In the relatively short time I used it, I ran into literally no bugs or issues, and the attempt to accommodate onboarding VS Code and other editor users was a really nice touch. Pretty much all of my reasons for not sticking with Zed were missing features rather than anything else.

So now let's talk about those missing features.

.NET is good, but missing Razor support

First, editing .NET, for the most part, just worked. I mean, it's like working in VS Code, which is basically all I wanted or needed. I'm not looking to replace Rider with Zed, so I don't need a full IDE.

The one piece that was missing is that there doesn't seem to be any support for Razor or cshtml files, which I *think* is an Omnisharp limitation. And if that's the case, it'll probably *never* be there unfortunately☹️


Rendering\Pages\About.razor

```
<section class="text-zinc-600 dark:text-zinc-300">
    <article class="pt-20 bg-white dark:bg-zinc-800 relative shadow-lg">
        @if (HasLogo)
        {
        <header class="-top-[75px] absolute image w-full">
            <img src="@Logo" alt="@ShortTitle avatar" class="mx-auto max-w-[150px] border-2 box-content rounded-full border-primary"/>
        </header>
        }
        <h1 class="mt-0 xs:mt-4 mb-4 break-words dark:text-zinc-300 font-light leading-tight text-4xl text-center text-zinc-700 tracking-tight">@Context.Title</h1>
        <section class="lg:px-10 md:p-6 p-2 sm:p-4 break-words prose prose-zinc prose-img:mx-auto prose-img:mx-auto prose-table:block prose-table:overflow-auto dark:prose
            @Context.ContentPage.Html
        </section>
    </article>
</section>

<SectionContent SectionName="@Sections.MetaAndCriticalCss">
    <RequiredLinkMetadata AbsoluteUrl="@Context.AbsoluteUrl" SiteContext="@Context.SiteContext" />
    <SiteGraphMetadata AbsoluteUrl="@Context.AbsoluteUrl" Title="@Context.Title" SiteContext="@Context.SiteContext" />
</SectionContent>

@code {

    public const string MainClass = "-top-6 xs:mx-4 sm:max-w-xl sm:mx-auto md:max-w-2xl lg:max-w-4xl relative block";

    [Parameter] public required StandalonePageContext Context { get; set; }

    string Logo => Context.SiteContext.Settings.Site.Logo;
    string ShortTitle => Context.SiteContext.Settings.Site.ShortTitle;
    bool HasLogo => !string.IsNullOrEmpty(Logo);
}
```

This isn't really a knock against Zed or the Omnisharp project; Razor is a notoriously difficult format to handle as it's *multiple* languages in one: HTML, C#, Blazor `@` syntax, CSS, JavaScript etc. And it's not really a big deal for me; Zed is meant to be my editor and I'm generally not going to be messing with Razor pages in an editor.

So overall, Zed is "good enough" with .NET to be my go-to editor. Unfortunately, that's not the case for Markdown.

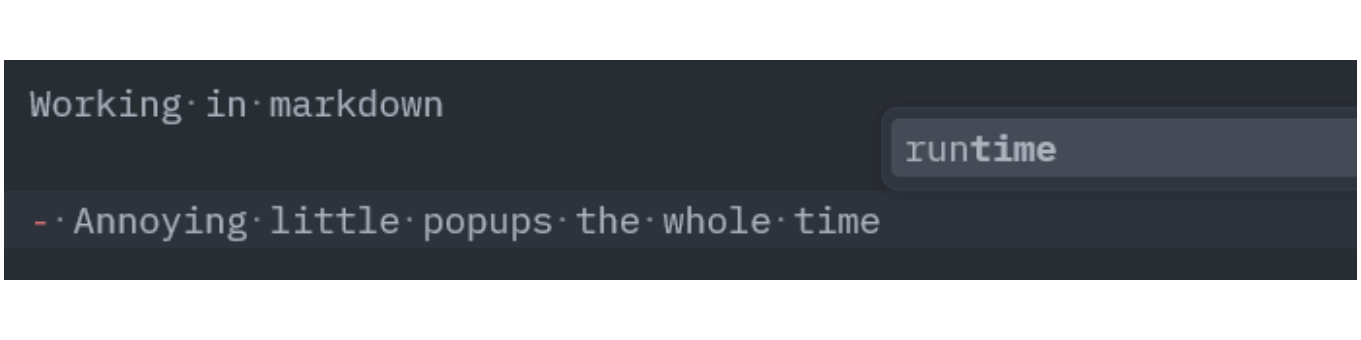
Markdown is missing too many features

After checking out .NET I was pretty optimistic with Zed, as I assumed that would be the hard bit, but unfortunately I was ultimately let down by the Markdown support.

Markdown has first-party support in Zed, but it's missing a lot of features that I'm used to having when writing in VS Code. Many of these are features provided by the extension I use, but they make for a very smooth writing experience:

- Shortcuts like `Ctrl`+`B` to bold the current word. This has been requested for Zed, but doesn't currently exist.
- "Code folding" for headings and other elements. These let you hide sections, just like you would when folding a method in code.
- "IntelliSense" for internal links and for images (which are added via a shortcut of course!).
- Images are rendered in the preview pane
- As you scroll the source page (without moving the cursor), the preview window scrolls too.

However, the biggest issue with markdown is the way Zed *constantly* pops up suggestions for the word you're currently writing. This is incredibly distracting, provides no value when writing markdown documents, and frankly made it unusable for me.



```
Working·in·markdown

-·Annoying·little·popups·the·whole·time
```

These all seem like small things, and they are, but they add up to a slick experience, and importantly an experience I'm *used* to. I've been writing this blog for long enough now that it's just not worth throwing away nearly a decade of muscle memory😅

That said, most of these are small features that absolutely *could* be added later or addressed, so I'm certainly going to keep an eye on it.

A lot of shortcuts don't port across

On the theme of muscle memory, one of the big struggles I had initially was realising *quite* how many shortcuts I use instinctively. This can be very confusing when you hit a shortcut expecting it to (for example) create a new cursor on the line below, and instead it switches to a completely different document😅

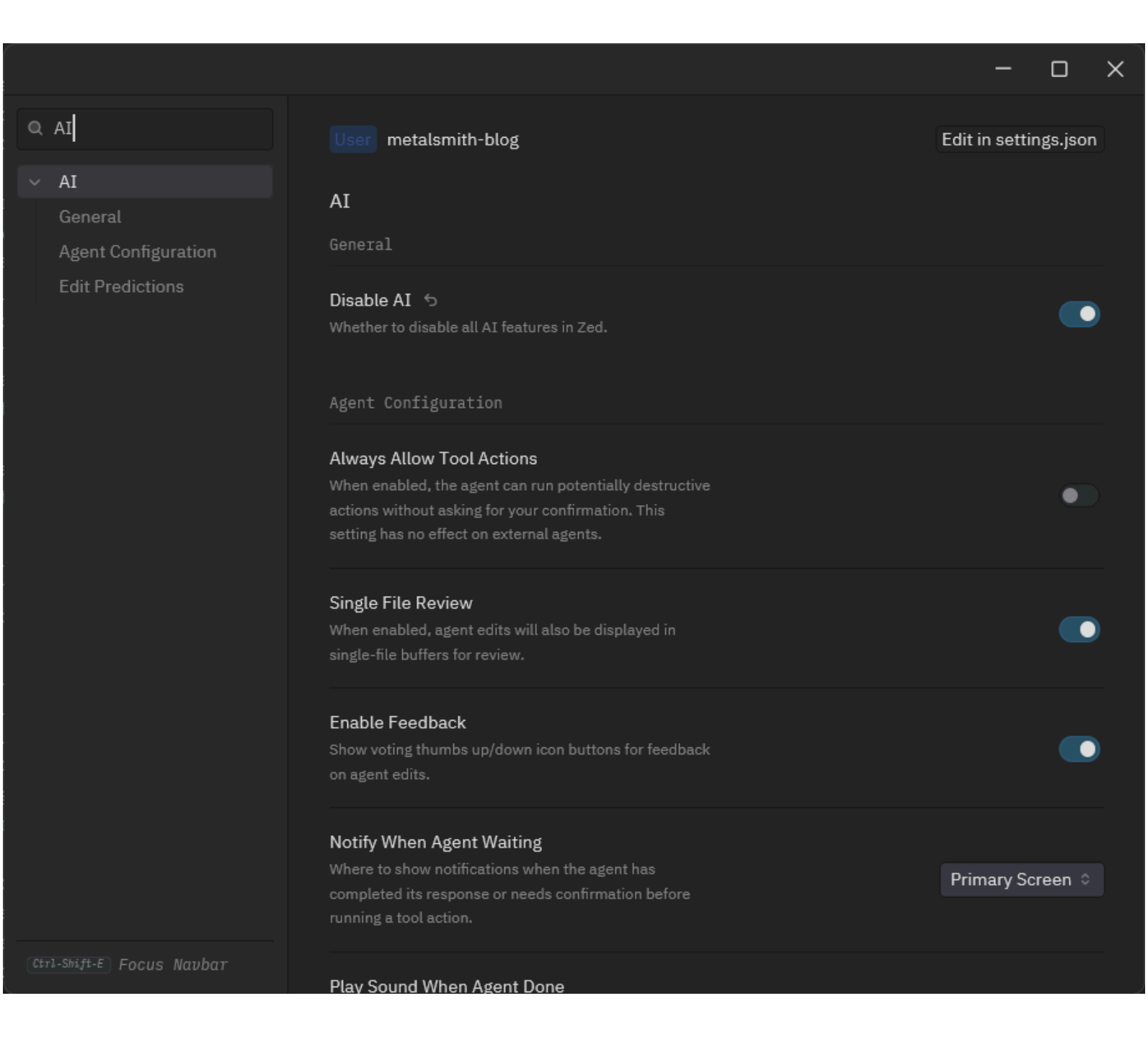
None of this is a big problem, as you can customize basically all of the keyboard shortcuts. I'm also not *entirely* sure if these were just not mapped in the "import Keymaps from VS Code" stage, or if these aren't part of the "core" VS Code or something. Anyway, I've added the following mappings so far to get something closer to what I have in VS Code:

```
// Zed keymap
[
  {
    "context": "Workspace",
    "bindings": {
      // "shift shift": "file_finder::Toggle"
    }
  },
  {
    "context": "Editor && vim_mode == insert",
    "bindings": {
      // "j k": "vim::NormalBefore"
    }
  },
  {
    "context": "Editor",
    "bindings": { "ctrl-shift-delete": "editor::DeleteLine" }
  },
  {
    "bindings": { "ctrl-d": "editor::DuplicateSelection" }
  },
  {
    "bindings": { "ctrl-k ctrl-f": "editor::FormatSelections" }
  },
  {
    "context": "Editor",
    "bindings": { "ctrl-k ctrl-d": "editor::Format" }
  },
  {
    "context": "Editor",
    "bindings": { "alt-enter": "editor::ToggleCodeActions" }
  },
  {
    "context": "Editor",
    "bindings": { "shift-f12": "editor::FindAllReferences" }
  },
  {
    "context": "Editor",
    "bindings": { "f3": [ "editor::SelectNext", { "replace_newest": false } ] }
  },
  {
    "context": "Editor",
    "bindings": { "shift-f3": [ "editor::SelectPrevious", { "replace_newest": false } ] }
  },
  {
    "context": "Editor",
    "bindings": { "ctrl-alt-down": "editor::AddSelectionBelow" }
  },
  {
    "context": "Editor",
    "bindings": { "ctrl-alt-up": "editor::AddSelectionAbove" }
  },
  {
    "context": "Workspace",
    "bindings": { "alt-s": "workspace::ToggleLeftDock" }
  },
  {
    "context": "Pane",
    "bindings": { "alt-left": "pane::GoBack" }
  },
  {
    "context": "Pane",
    "bindings": { "alt-right": "pane::GoForward" }
  },
  {
    "context": "Editor",
    "bindings": { "ctrl-k ctrl-c": [ "editor::ToggleComments", { "advance_downwards": false } ] }
  }
]
```

Which brings us to the final point.

The AI elephant in the room

As I already mentioned, I don't want or need AI in my editor. And yet every company has to add AI to their product whether you want it or not. Such is the way of our times. VS Code and Visual Studio are the same. But the good thing about Zed is that you can simply disable it if you don't want it:


User metalsmith-blog

AI

General

Disable AI — Whether to disable all AI features in Zed. (on)

Agent Configuration

Always Allow Tool Actions — When enabled, the agent can run potentially destructive actions without asking for your confirmation. This setting has no effect on external agents. (off)

Single File Review — When enabled, agent edits will also be displayed in single-file buffers for review. (on)

Enable Feedback — Show voting thumbs up/down icon buttons for feedback on agent edits. (on)

Notify When Agent Waiting — Where to show notifications when the agent has completed its response or needs confirmation before running a tool action. Primary Screen

Play Sound When Agent Done

Alternatively you can edit all the settings as JSON. This is what I have currently:

```
// Zed settings
{
  "ensure_final_newline_on_save": false,
  "remove_trailing_whitespace_on_save": false,
  "format_on_save": "off",
  "when_closing_with_no_tabs": "keep_window_open",
  "disable_ai": true,
  "telemetry": {
    "diagnostics": true,
    "metrics": true
  },
  "project_panel": {
    "auto_fold_dirs": false
  },
  "base_keymap": "VSCode",
  "minimap": {
    "show": "never"
  },
  "file_types": {},
  "show_whitespaces": "trailing",
  "icon_theme": {
    "mode": "dark",
    "light": "Zed (Default)",
    "dark": "JetBrains New UI Icons (Dark)"
  },
  "ui_font_size": 13.0,
  "buffer_font_size": 12.0,
  "theme": {
    "mode": "dark",
    "light": "One Light",
    "dark": "JetBrains Rider Dark"
  }
}
```

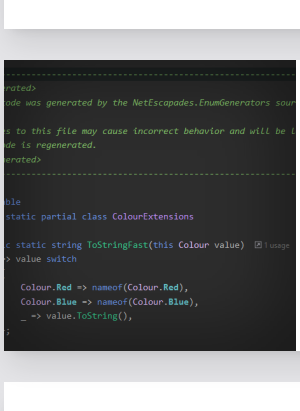
And that's pretty much it. In conclusion, I won't be replacing VS Code with Zed today. But I can definitely see a time in the near future where enough of the missing markdown features I'm used to have been added that I'll make the switch. The Zed team are shipping updates regularly, and they have a really nice product, even if it's not quite there for me.

Summary

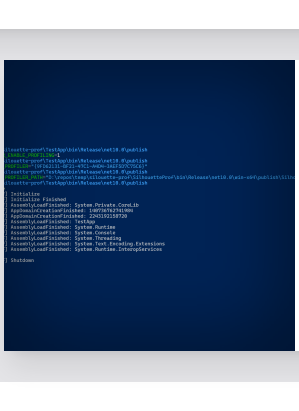
In this post I described my experience installing the Zed editor, and trying to replace my VS Code usages with Zed. In particular, I wanted to use Zed to edit .NET projects and to write markdown documents. The .NET experience was very similar to the VS Code experience with the Omnisharp plugin, which is mostly only lacking with regards to Razor/Blazor support. For Markdown I found the lack of features I'm used to in VS Code to be a deal breaker. Zed is an incredibly fast and smooth experience, but I won't be switching to it currently. If the features I'm missing are added however, I would seriously consider it.

FOLLOW ME

ENJOY THIS BLOG?



PREVIOUS
Recent updates to NetEscapades.EnumGenerators: [EnumMember] support, analyzers, and bug fixes



NEXT
Creating a .NET CLR profiler using C# and NativeAOT with Silhouette

© 2026 Andrew Lock | .NET Escapades. All Rights Reserved. | Image credits